THE STACK WE ARE BUILDING

Think of your AI as a stack. At the bottom sits the model you already know how to run. Above it sits a model interface handling the correct chat template and generation settings. Next comes an application layer deciding what happens for each request. Beside it sit knowledge stores containing documents and embeddings, plus tools connected to functions, databases or APIs.

A memory layer retains state where appropriate. An evaluation layer measures whether the complete pipeline works. Finally, the user interface hides that complexity behind a chat window, form, dashboard or specialised workflow.

You do not replace the model every time you add a capability. You improve the layers around it. That modularity is one of the biggest advantages of treating AI as a system rather than a single model file.

Changing the model will not repair a broken search index or an unreliable tool.

This is why the rest of this Fast Track focuses heavily on the surrounding system.

Build a workspace you can reuse

Before moving into data preparation, RAG and fine-tuning, create a clean project that can grow with you. The exact commands vary by operating system, but the structure should remain simple.

Step 1: Create a project folder. Make a directory called `my-ai` and keep everything related to this build inside it. Avoid scattering scripts and datasets across Downloads, Desktop and random notebooks.

Step 2: Create a Python virtual environment. From a terminal inside the folder, run `python -m venv .venv`. Activate it using the appropriate Windows, macOS or Linux command. A virtual environment prevents libraries for this project from colliding with packages used elsewhere.

Step 3: Create five working folders. Add `data`, `src`, `models`, `tests` and `outputs`. Put source documents and datasets in `data`; application code in `src`; adapters or model artefacts in `models`; evaluation cases in `tests`; and generated reports in `outputs`.

Your directory should resemble:

```
my-ai/
├── data/
├── models/
├── outputs/
├── src/
├── tests/
└── .venv/
```

Step 4: Add a configuration file. Create a `.env` file for local configuration values such as model endpoints, database paths or API keys used later. Do not hard-code secrets into Python files, and do not commit the `.env` file to a public repository.

Step 5: Record dependencies. As you install packages, maintain a `requirements.txt` or equivalent environment file. Reproducibility becomes important surprisingly quickly. "It worked on my laptop last week" is not a deployment strategy.

Step 6: Create a minimal application entry point. Add `src/app.py`. Initially it can contain nothing more than a function that sends a message to your local model endpoint and prints the response. The point is to establish one predictable place from which the later pipeline will grow.

Step 7: Create your first evaluation file now. In `tests`, add a small text or JSON file containing five prompts your current setup should handle correctly. This feels premature, but it gives you a baseline. As retrieval, tools and fine-tuning are added, rerun the same questions and see whether the system actually improved.

Add dependencies only when the project needs them. A comprehensible system is easier to debug than a fashionable, overloaded stack.

Your first architecture sketch

Before the next chapter, choose one AI you genuinely want to build. Keep it narrow. "An AI that does everything" is not a project specification.

Write one sentence describing the user and task. For example: "A private assistant that lets me ask questions across my work documents and always points to the source." Then write down four things: what information it needs, what behaviour you expect, what external actions it must perform, and how you will know an answer is correct.

That exercise tells you which layers matter. The private-document assistant needs retrieval and citations. It probably does not need fine-tuning on day one. A ticket classifier may need structured output and eventually an adapter, but perhaps no external tools. A local research assistant may need RAG, Python tools and a stronger evaluation set.

This architecture-first habit will save you from building technology merely because it is interesting.

You already own the hardest component in the stack: a capable model that runs under your control. The next stage is not about making that model bigger. It is about making the system around it smarter—feeding it the right information, teaching it the right behaviour, connecting it to useful capabilities and measuring whether the combination deserves to be trusted.

The model was the beginning. Now we build the AI.